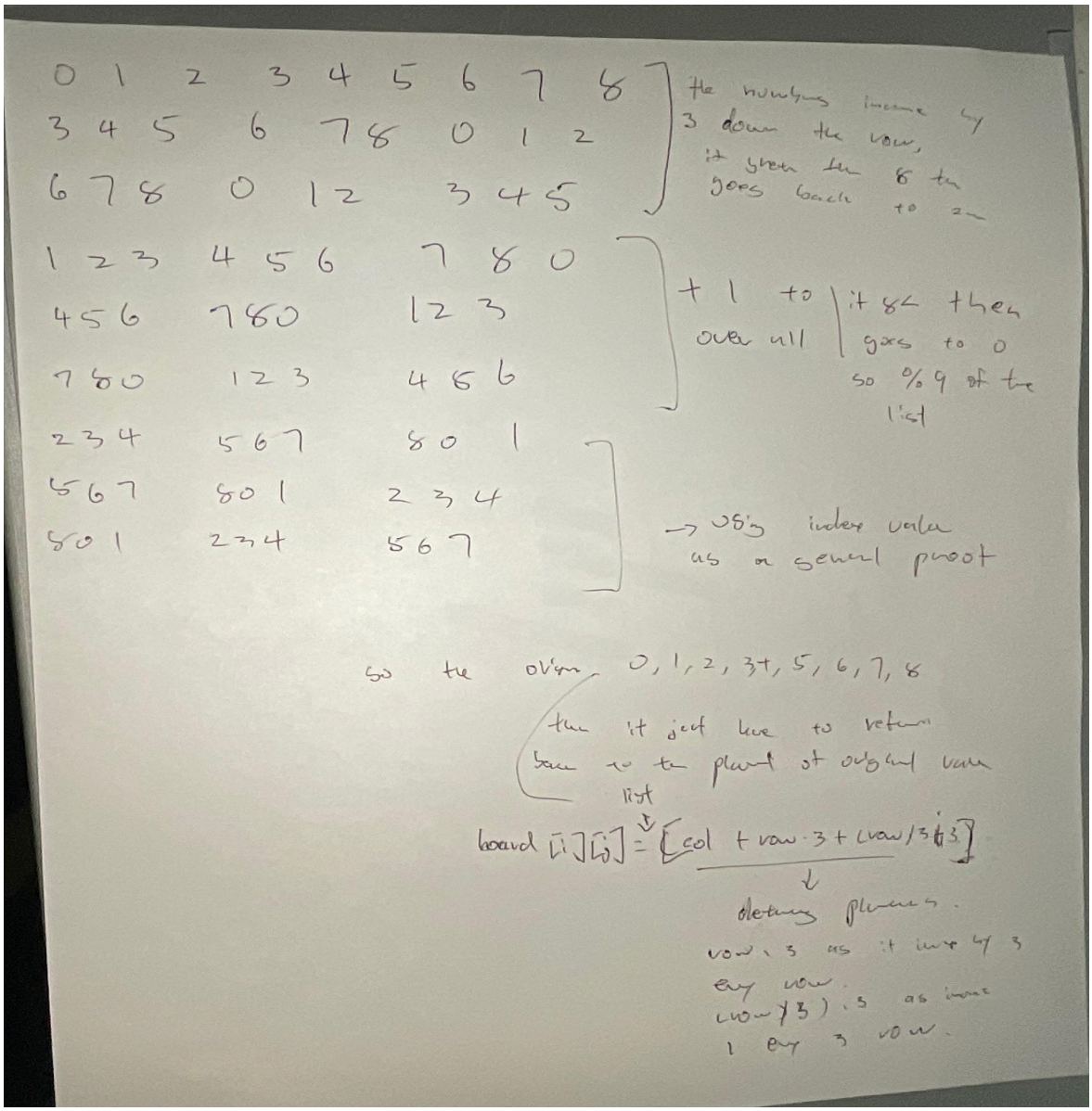
My code creates a typical 9x9 sudoku board where each row, each column, each 3x3 sub grid contains a list of 1 - 9 without repeating. First of all I set up a 2d array where it is 9 by 9. Then I have the method which shuffles my array of 1 - 9 for the first column so I have a random sudoku every time. Then based on the first row I generated I used a math equation that I proved to refer back to what number in the first column would be a fittable number for the current position in the board. With a nested for loop that repeats for the whole board it would successfully create a 2d array list with proper numbers in each spot. Then I have the final method which prints out the board nicely. In the main I first generate the board and then print it out nicely to create the 9x9 sudoku.

So I have 2 arrays that I created for the sudoku. One the 2d array for sudoku itself as it has both rows and columns. The other one is to create the list for the first column that contains shuffled 1-9 as my reference for the rest of sudoku.

I use the 2d array as the storage for every value of the numbers in terms of points where the row is the y axis and column is the x axis. Through using my equation then storing the value into the specific column and axis, I then at the end print it out in the same format as the output shown in the google doc.

I use the 1d array to create the shuffled first column. First I generate a list of 1-9, then using a method I create which acts as a shuffle it would randomly shuffle the list so it creates a random first column. Then because the rest of the values refers back to the order of this column it not only helps as an index for the rest of the values but also creates a random sudoku everytime instead of a hardcoded first line making the same sudoku for every trial.



So I was looking in the program output sample in the google doc and I realized a pattern. I found out the rest of the board seems to be shifting in a way that refers back to the first column. Then I started to proof it on paper using 0-8 as general index value rather than specific values to prove that the pattern works for all cases rather than a single sudoku.

If you go down by rows, you will realize that the numbers increase by 3 for every row you went down and an additional 1 row every 3 rows. When a placement adds up and exceeds 8

it goes back to the first value in the original list. The column is however much simpler as it just increases by 1 every next value and goes back to 0 when it exceeds 8. This makes me conclude up to the solution that the placement is (column + row*3 + row/3*3)%9 in the first column. The column is simply because it increases as column increase, the row 3 is because for every row down it increase by 3 to the value and the row/3*3 is because for every 3 rows it increases an additional number. Lastly, the %9 indicates it goes back to 0 in the list after exceeding the length of the original list which is equal to 9.

The first method is called shuffle, like its name it acts as a shuffler using the fisher yates method to the col array that I created containing 1-9 making it a random order list of 1-9 ensuring that I have a different sudoku every time.

The second method is the generateBoard method which assigns value to the 2d array for sudoku. It first creates the list 1-9 then shuffles it using the method I just talked about. Then through a nested for loop it assigns the value of each index based on the equation i provided.

Lastly is the printBoard method which prints out the 2d array in a way visually appealing. In this method it follows the sample output and prints it in the same way, separating it to 9 small grids and closing the outside.'

For testing I not only checked over my proof on paper but I also ran the program multiple times (more than 3) and checked whether or not the program's generated sudoku actually fits in the rule. As both my proof and my output for all trails demonstrate a valid sudoku, I was finally sure my code works.

One challenge was figuring out what was the pattern to sudoku board. I haven't played sudoku for a long time and was not really familiar with it. I really struggled with whether or not sudoku has a pattern. Then through the sample output when I was scrolling through the

document for inspiration I realized that it does. Then through assigning the index and proofing how there is a way to generate sudoku through a certain order based on the first column, I successfully derived the equation and moved on in my code conquering great obstacles.